



1.1. Introducción

Desarrollo de Aplicaciones Web, segundo

© por Sergio Montes Repiso

Contenidos curriculares y resultados de aprendizaje

Selección de arquitecturas y tecnologías de programación:

- Mecanismos de ejecución de código en un navegador web.
- Capacidades y limitaciones de ejecución.
- Lenguajes de programación en entorno cliente.
- Tecnologías y lenguajes asociados.
- Integración del código con las etiquetas HTML.
- Herramientas de programación y prueba sobre clientes web. Librerías y frameworks.

RA1. Selecciona las arquitecturas y tecnologías de programación sobre clientes Web, identificando y analizando las capacidades y características de cada una.

Criterios de Evaluación

- a) Se han caracterizado y diferenciado los modelos de ejecución de código en el servidor y en el cliente Web.
- b) Se han identificado las capacidades y mecanismos de ejecución de código de los navegadores Web.
- c) Se han identificado y caracterizado los principales lenguajes relacionados con la programación de clientes Web.
- d) Se han reconocido las particularidades de la programación de guiones y sus ventajas y desventajas sobre la programación tradicional.
- e) Se han verificado los mecanismos de integración de los lenguajes de marcas con los lenguajes de programación de clientes Web.
- f) Se han reconocido y evaluado las herramientas de programación sobre clientes Web.

¿Qué vamos a aprender?

O1

Modelos de Ejecución

Diferencias entre ejecución en servidor y cliente web

O2

Capacidades del Navegador

Mecanismos de ejecución de código en navegadores

O3

Lenguajes Cliente

Principales tecnologías de programación web

O4

Programación de Scripts

Ventajas y limitaciones de los lenguajes de guiones

O5

Integración HTML

Combinación de marcado y programación

O6

Herramientas y Frameworks

Entorno de desarrollo y librerías modernas

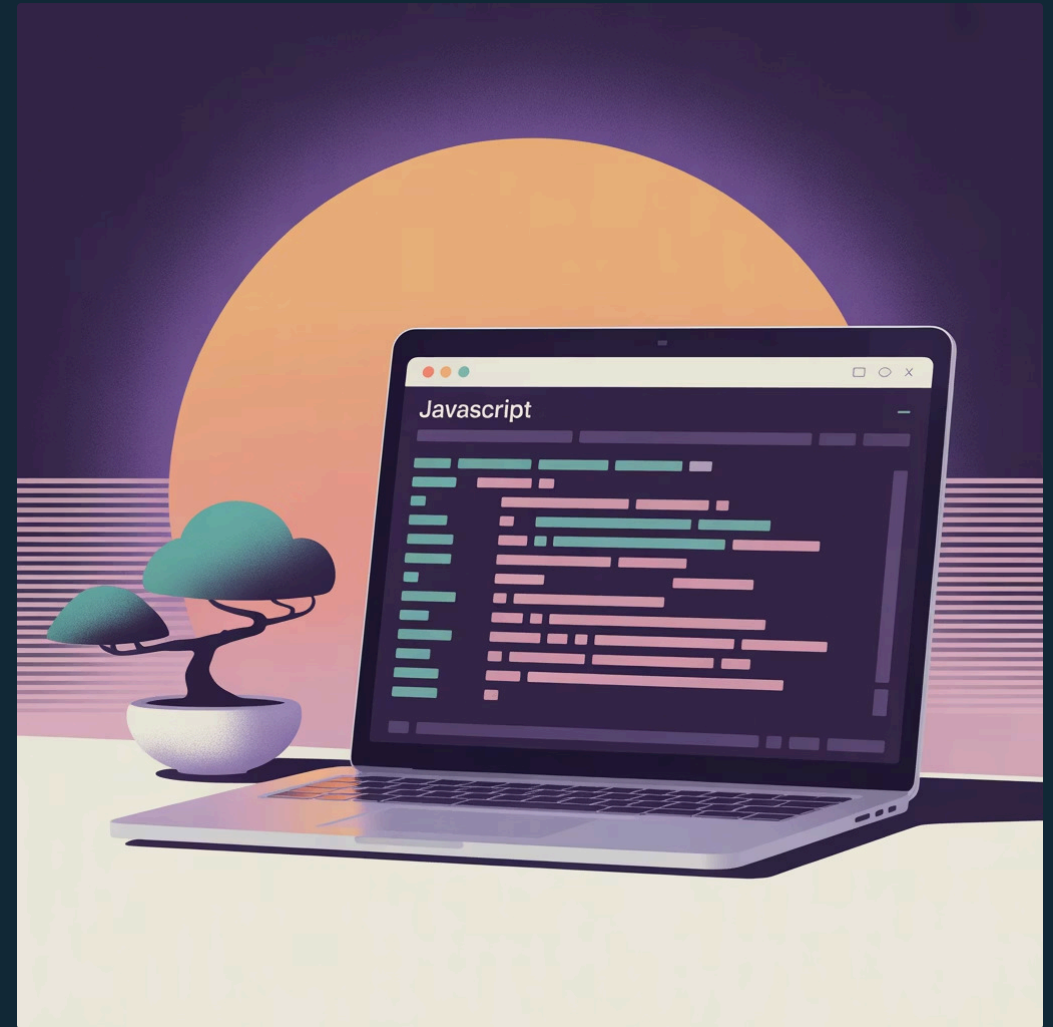
Modelos de Ejecución: Servidor vs Cliente

Ejecución en el Servidor



- El código se ejecuta antes del envío al navegador
- El cliente recibe HTML estático ya procesado
- Mayor seguridad y control de datos
- Requiere comunicación constante con el servidor

Ejecución en el Cliente



- El código se ejecuta directamente en el navegador
- Interactividad inmediata sin recarga de página
- Reduce la carga del servidor
- Dependiente de las capacidades del navegador

El Navegador como Plataforma de Ejecución

Los navegadores modernos han evolucionado hasta convertirse en potentes plataformas de ejecución, capaces de interpretar y ejecutar código complejo directamente en el dispositivo del usuario.



Motor JavaScript

V8 (Chrome), SpiderMonkey (Firefox), JavaScriptCore (Safari) procesan el código JavaScript de forma optimizada.



DOM y APIs

Interfaz de programación para manipular elementos HTML, CSS y acceder a funcionalidades del sistema.



Entorno Seguro

Sandbox que aísla la ejecución del código para proteger el sistema operativo del usuario.

INFO EXTRA

Cómo funcionan los motores

El proceso simplificado funciona así:

1. **Parser (Analizador):** El motor lee el código y lo convierte en una estructura de árbol llamada **AST** (Árbol de Sintaxis Abstracta).
2. **Intérprete básico:** El motor empieza a ejecutar el código rápidamente transformándolo en un *bytecode* intermedio.
3. **Perfilado (Profiler):** Mientras el código corre, el motor vigila qué partes se ejecutan más veces (el código "caliente" o *hot code*). Por ejemplo, un bucle que se repite 10,000 veces.
4. **Compilador optimizador:** El motor toma ese código "caliente" y lo compila directamente a código máquina ultrapotente, haciendo suposiciones para ganar velocidad. Si esas suposiciones fallan (desoptimización), vuelve al intérprete básico.

El fenómeno de "Bun" (Basado en JavaScriptCore)

Aunque técnicamente es un *entorno de ejecución* (runtime) para servidores —como **Node.js**— y no un motor creado desde cero, **Bun** merece una mención obligatoria por el impacto que ha causado.

- **El giro moderno:** A diferencia de Node.js y Deno (que usan el motor V8 de Chrome), Bun decidió romper el molde y utilizar **JavaScriptCore** (el motor de Safari).
- **¿Por qué es más rápido?:** JavaScriptCore tiene un tiempo de arranque (*startup time*) drásticamente menor que V8 y está programado en **Zig** (un lenguaje de programación moderno de bajo nivel). Bun aprovecha esto para crear un entorno de servidor enfocado en aplicaciones *Serverless* y microservicios, logrando velocidades de ejecución y de instalación de paquetes que duplican o triplican a Node.js.

Sandbox

Su objetivo es simple pero vital: **asumir que todo código que descargas de internet es potencialmente peligroso** y ejecutarlo en un entorno completamente aislado para que no pueda dañar tu computadora.

Cuando entras a una página web, tu navegador descarga automáticamente archivos de JavaScript y los ejecuta en tu computadora. Si no hubiera un Sandbox, un hacker podría escribir un script en su página web que, al solo visitarla, hiciera cosas como leer tus archivos privados, instalar virus, ...

Capacidades y Limitaciones del Entorno Cliente

Capacidades Principales

- **Manipulación del DOM:** Modificar estructura y contenido de la página en tiempo real.
- **Gestión de eventos:** Responder a interacciones del usuario (clicks, formularios, scroll).
- **Comunicación asíncrona:** Realizar peticiones al servidor sin recargar la página (AJAX).
- **Almacenamiento local:** Guardar datos en el navegador (localStorage, sessionStorage).
- **APIs modernas:** Geolocalización, cámara, notificaciones, service workers.

Limitaciones Importantes

- Acceso restringido al sistema de archivos
- Política del mismo origen para seguridad
- Dependencia de la potencia del dispositivo cliente





Lenguajes de Programación en Entorno Cliente

1

JavaScript

El lenguaje principal para programación cliente. Dinámico, interpretado y con soporte nativo en todos los navegadores modernos.

2

TypeScript

Superset de JavaScript que añade tipado estático. Se transpila a JavaScript para ejecutar en el navegador.

3

WebAssembly (WASM)

Formato binario que permite ejecutar código compilado desde otros lenguajes (C++, Rust) en el navegador.

4

Dart

Lenguaje de Google, principalmente usado con Flutter para desarrollo web, que se transpila a JavaScript.



Compile vs. Transpile

A veces se confunden, pero tienen objetivos muy distintos:

- **Compile (De alto nivel a bajo nivel):** Toma un lenguaje comprensible por humanos (como C++ o Java) y lo traduce a un lenguaje que solo entiende la máquina (código binario de 0s y 1s, o *bytecode*). Es un viaje "hacia abajo".
- **Transpile (De alto nivel a alto nivel):** Toma un lenguaje legible por humanos y lo traduce a **otro** lenguaje también legible por humanos. Es un viaje "hacia los lados".

JavaScript: El Corazón del Desarrollo Cliente

JavaScript es el lenguaje fundamental para el desarrollo web cliente. Su evolución desde un simple lenguaje de scripts hasta una plataforma completa de desarrollo ha sido extraordinaria.

Características Principales

- **Interpretado:** No requiere compilación previa
- **Dinámico:** Tipos de datos flexibles
- **Orientado a eventos:** Responde a acciones del usuario
- **Funcional y orientado a objetos:** Múltiples paradigmas
- **Asíncrono:** Manejo de operaciones no bloqueantes.

Las versiones modernas (ES6+) han introducido características avanzadas como arrow functions (funciones flecha), async/await, modules y destructuring.



Programación de Scripts vs Programación Tradicional



Programación de Scripts

Ventajas: Desarrollo rápido, interpretación directa, flexibilidad de tipos, ideal para automatización y prototipado.

Desventajas: Menor rendimiento, detección de errores en tiempo de ejecución, dependencia del intérprete.



Programación Tradicional

Ventajas: Mayor rendimiento, detección temprana de errores, optimización del compilador, ejecución independiente.

Desventajas: Proceso de compilación, menor flexibilidad, desarrollo más lento para cambios rápidos.

En el desarrollo web moderno, estas diferencias se han difuminado con herramientas como transpiladores y bundlers que combinan lo mejor de ambos enfoques.



¿Alguien puede decir por qué un lenguaje de script tiene peor rendimiento que uno tradicional?

Integración de Código con HTML

La integración efectiva entre HTML y JavaScript es fundamental para crear aplicaciones web dinámicas e interactivas.

Scripts Inline

Código JavaScript directamente en atributos HTML (onclick, onload). Útil para funcionalidades simples pero no recomendado para proyectos complejos.

Scripts Externos

Archivos JavaScript separados referenciados desde HTML. Mejora la organización, reutilización y mantenimiento del código.

1

2

3

4

Scripts Internos

Código JavaScript dentro de etiquetas `<script>` en el documento HTML. Mantiene la lógica en el mismo archivo.

Módulos ES6

Sistema moderno de importación/exportación que permite organizar el código en módulos reutilizables y mantener un namespace limpio.

Ejemplo de Integración

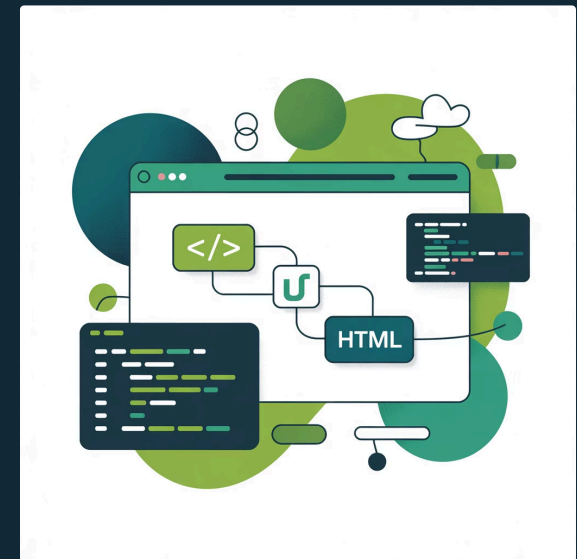
```
<!DOCTYPE html>
<html>
<head>
  <title>Integración HTML-JS</title>
</head>
<body>
  <button id="miBoton">Hacer clic</button>
  <p id="mensaje"></p>

  <script>
    const boton = document.getElementById('miBoton');
    const mensaje = document.getElementById('mensaje');

    boton.addEventListener('click', function() {
      mensaje.textContent = 'Botón pulsado!';
    });
  </script>
</body>
</html>
```

Buenas Prácticas

- Separar estructura (HTML), presentación (CSS) y comportamiento (JS)
- Usar `addEventListener` en lugar de atributos `onclick`
- Cargar scripts al final del body o usar `defer/async`
- Validar la existencia de elementos antes de manipularlos



INFO EXTRA

La solución moderna: Módulos (import y export)

Con el sistema de módulos, cada archivo de JavaScript se convierte en una **cápsula privada y aislada** (un Sandbox para ese archivo). Nada de lo que escribas dentro de `archivo1.js` existe para el resto del mundo, **a menos que tú decidas exportarlo explícitamente**.

Esto te permite controlar con precisión milimétrica qué partes de tu código compartes y cuáles mantienes ocultas.

Ejemplo Real de cómo funciona:

Imagina que tienes un archivo externo llamado `calculadora.js`:

```
// archivo: calculadora.js

// Esta variable es PRIVADA. Nadie fuera de este archivo puede verla ni alterarla.
const configuracionInterna = "secreto123";

// Con la palabra 'export', haces que esta función sea PÚBLICA para otros archivos.
export function sumar(a, b) {
  return a + b;
}
```

Ahora, en tu archivo principal (`main.js`), puedes traer **únicamente** lo que necesitas:

```
// archivo: main.js

// Usas 'import' para traer la función específica del otro archivo externo
import { sumar } from './calculadora.js';

console.log(sumar(5, 10)); // Imprime 15

// Si intentas hacer esto, dará un ERROR porque no fue exportado:
// console.log(configuracionInterna);
```

¿Qué ventajas reales te da esto en el día a día?

1. **Mantener un namespace limpio:** Puedes tener una función llamada `configurar()` en 5 archivos distintos. Como cada uno es un módulo aislado, jamás se van a pisar ni a generar conflictos entre sí.
2. **Código reutilizable:** Creas un módulo para conectarte a una base de datos o validar un email una sola vez, y simplemente lo *importas* en los 20 lugares de tu aplicación donde lo necesitas.
3. **Carga inteligente (Optimización):** Los navegadores y herramientas modernas pueden leer tus `imports` y descargar solo los archivos que realmente se están usando en esa pantalla, haciendo que la web cargue mucho más rápido.

En resumen

- **Archivo externo:** Es simplemente sacar el código del HTML y meterlo en otro documento.
- **Módulos (import/export):** Es ponerle "puertas y cerraduras" a esos archivos externos para que compartan información solo cuando tú lo autorizas, evitando que el código se mezcle y se rompa.

Herramientas de Desarrollo Web Cliente



Editores de Código

Visual Studio Code (y todos sus forks), WebStorm, Sublime Text (si tenéis narices de usarlo). Ofrecen resaltado de sintaxis, autocompletado, debugging integrado y extensiones específicas para desarrollo web.



Developer Tools

Herramientas integradas en navegadores (F12) para inspeccionar elementos, debuggear JavaScript, analizar rendimiento y monitorizar red.



Gestores de Paquetes

npm, yarn, pnpm para instalar y gestionar dependencias. Facilitan la integración de librerías externas y herramientas de desarrollo.

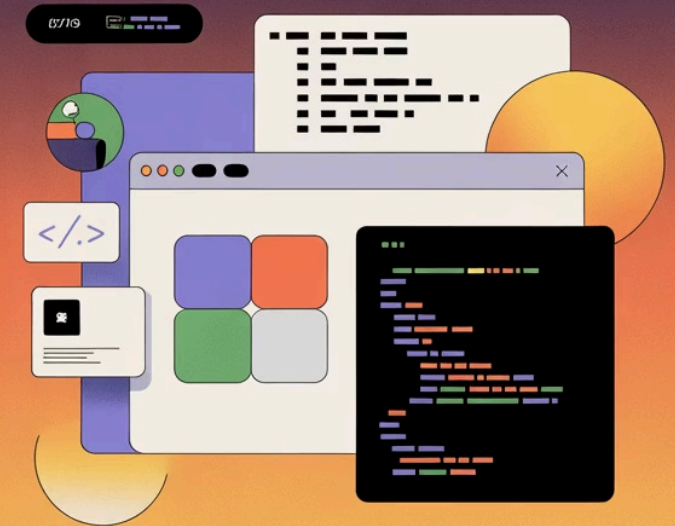


Build Tools

Webpack, Vite, Parcel para bundling, minificación, transpilación y optimización automática del código para producción.

Codecraft: Your Development Toolkit

Get started



Librerías y Frameworks Modernos

El ecosistema JavaScript ofrece múltiples opciones para acelerar el desarrollo y mejorar la mantenibilidad del código.

Librerías de Utilidad

- **jQuery:** Simplifica manipulación DOM y AJAX
- **Lodash:** Funciones útiles para manipulación de datos
- **Axios:** Cliente HTTP para peticiones al servidor

Frameworks SPA

- **React:** Biblioteca para interfaces de usuario basada en componentes
- **Vue.js:** Framework progresivo fácil de integrar
- **Angular:** Plataforma completa para aplicaciones empresariales

Frameworks CSS

- **Bootstrap:** Framework responsive para diseño rápido
- **Tailwind CSS:** Utilidades CSS para diseño personalizado
- **Bulma:** Framework CSS moderno basado en Flexbox

📌 El matiz exacto: React vs. Frameworks

- **React (Librería):** Tú la llamas a ella. Te da libertad absoluta para elegir cómo gestionas las rutas, el estado global o los formularios. El desarrollador mantiene el control de la arquitectura arquitectónica.
- **Angular y Vue (Frameworks):** Ellos te llaman a ti. Te imponen una estructura de carpetas, un sistema de enrutado nativo y una forma estricta de trabajar.

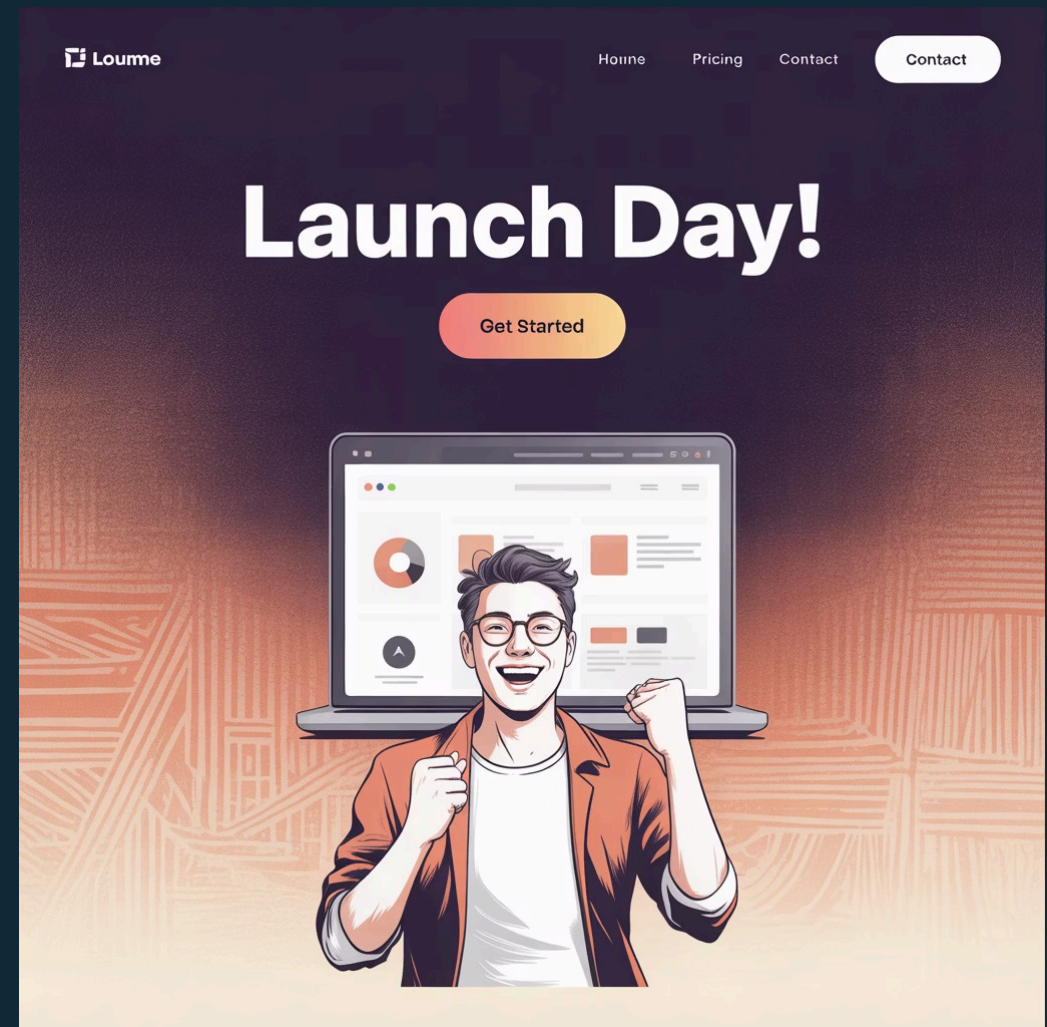
La razón por la que he metido React como framework es porque habitualmente se suele trabajar dentro de un framework (por ejemplo, next.js) , no la librería de por sí sola.

- **Nuestra ruta:** Comenzar con JavaScript vanilla y progresivamente incorporar librerías y/o frameworks según las necesidades del proyecto. ¡Hay que aprender a andar antes de echarse a correr!

Resumen: Dominando el Desarrollo Cliente

Conceptos Clave Aprendidos

- Diferencias fundamentales entre ejecución servidor/cliente
- Capacidades y limitaciones de los navegadores modernos
- JavaScript como lenguaje principal del entorno cliente
- Ventajas de la programación de scripts en el contexto web
- Métodos de integración entre HTML y JavaScript
- Herramientas esenciales para desarrollo profesional



"El desarrollo web cliente es la puerta de entrada a crear experiencias digitales que conectan directamente con los usuarios. Dominar estas tecnologías te permitirá construir interfaces dinámicas, interactivas y profesionales."